图像分类实验报告

深度学习课程 · 实验一

1.1 实验目的

本实验旨在通过在 CIFAR-10 和 CUB-200-2011 两个基准数据集上完成图像分类任务，达到以下学习目标：

- 理解卷积神经网络（CNN）的基本原理和各层次特征提取机制。
- 掌握使用 PyTorch 构建、训练和评估深度学习模型的完整工程流程。
- 熟悉主流网络架构（ResNet、AlexNet、VGG、MobileNet、ViT）的结构特点及适用场景。
- 深入理解注意力机制（Attention），并能够从零手动实现 SE（Squeeze-and-Excitation）模块，构建 SENet 并验证其效果。
- 掌握数据增强策略对模型泛化能力的影响，以及迁移学习在细粒度分类中的应用方式。

1.2 实验要求

- 基于 CIFAR-10 数据集（10 类，共 60000 张 32×32 彩色图像）完成基础多分类任务。
- 基于 CUB-200-2011 数据集（200 类鸟类，共 11788 张图像）完成细粒度图像分类任务。
- 选取不少于两种不同的网络架构进行对比实验，分析各自的优缺点。
- 在不借助任何现成 SENet 实现的前提下，完全手动实现 SE 注意力模块，并将其集成到 ResNet 主干中。
- 对实验结果进行充分分析，包括训练曲线、混淆矩阵、模型对比等。

1.3 实验环境

- 操作系统：macOS Darwin
- 编程语言：Python 3.x
- 深度学习框架：PyTorch（含 torchvision）
- 主要依赖：numpy、matplotlib、PyYAML、tqdm
- 计算平台：运行时自动检测 CUDA / MPS / CPU

1.4 实验内容

1.4.1 数据集与预处理

CIFAR-10 使用 torchvision 内置接口加载，训练集采用 Resize(64) → RandomHorizontalFlip → RandomCrop(64, padding=8) → Normalize(mean=(0.4914, 0.4822, 0.4465), std=(0.2023, 0.1994, 0.2010)) 的增强管道；验证集仅做中心裁剪和归一化。

CUB-200-2011 需要自定义 Dataset 类，依据官方提供的 images.txt、image_class_labels.txt、train_test_split.txt 三份索引文件构建训练/测试分割。训练增强采用 Resize(256) → RandomHorizontalFlip → ColorJitter → CenterCrop(224) → Normalize(ImageNet 均值/方差)，以充分利用预训练权重的特征迁移能力。

1.4.2 网络架构

所有模型均替换原始分类头（最后的全连接层），使其输出维度与目标类别数匹配（CIFAR-10: 10 类；CUB: 200 类）。加载 ImageNet 预训练权重（ViT、ResNet、AlexNet、VGG、MobileNet），仅 SENet 因架构修改而随机初始化。

1.4.3 手写 SENet 实现

SE（Squeeze-and-Excitation）模块的核心思想是对每个通道做全局统计（Squeeze），再通过两层全连接学习通道间依赖关系，输出每个通道的重要性权重（Excitation），最后对原特征图做逐通道缩放（Re-calibration）。完整实现如下：

```
class SEBlock(nn.Module):
"""Squeeze-and-Excitation 注意力模块（手动实现）"""
def __init__(self, channels: int, reduction: int = 16) -> None:
super().__init__()
# Squeeze: 全局平均池化，将 H×W 压缩为 1×1
self.squeeze = nn.AdaptiveAvgPool2d(1)
# Excitation: 两层 FC 学习通道权重
self.excitation = nn.Sequential(
nn.Linear(channels, channels // reduction, bias=False),
nn.ReLU(inplace=True),
nn.Linear(channels // reduction, channels, bias=False),
nn.Sigmoid(),
)

def forward(self, x: torch.Tensor) -> torch.Tensor:
b, c, _, _ = x.size()
# Squeeze
y = self.squeeze(x).view(b, c)
# Excitation
y = self.excitation(y).view(b, c, 1, 1)
# Re-calibration: 逐通道加权
return x * y.expand_as(x)


class SEBasicBlock(nn.Module):
""" 集成 SE 模块的 ResNet BasicBlock"""
expansion = 1

def __init__(self, in_channels, out_channels, stride=1, reduction=16):
super().__init__()
self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
self.bn1 = nn.BatchNorm2d(out_channels)
self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
self.bn2 = nn.BatchNorm2d(out_channels)
self.se = SEBlock(out_channels, reduction)
self.relu = nn.ReLU(inplace=True)
self.downsample = (
nn.Sequential(nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
nn.BatchNorm2d(out_channels))
if stride != 1 or in_channels != out_channels else None
)

def forward(self, x):
identity = x
out = self.relu(self.bn1(self.conv1(x)))
out = self.bn2(self.conv2(out))
out = self.se(out) # ← SE 注意力
if self.downsample: identity = self.downsample(x)
return self.relu(out + identity)
```

1.4.4 训练配置

统一采用 Adam 优化器配合 StepLR 学习率调度策略。CIFAR-10 系列使用较大学习率（1e-3）和较小图像尺寸（64px）；CUB 系列使用较小学习率（1e-4）和标准尺寸（224px）以适配预训练特征。具体超参数如下表所示：

实验配置	优化器	学习率	Batch	图像尺寸	Epochs	学习率调度
CIFAR-10 ResNet-18	adam	0.001	256	64	10	StepLR(step=5, γ=0.5)
CIFAR-10 SENet	adam	0.001	128	64	10	StepLR(step=5, γ=0.5)
CIFAR-10 AlexNet	adam	0.001	128	64	10	StepLR(step=5, γ=0.5)
CIFAR-10 MobileNet	adam	0.001	128	64	10	StepLR(step=5, γ=0.5)
CIFAR-10 VGG-16	adam	0.001	128	64	10	StepLR(step=5, γ=0.5)
CUB ResNet-50	adam	0.0001	64	224	20	StepLR(step=8, γ=0.5)
CUB SENet	adam	0.0001	64	224	20	StepLR(step=8, γ=0.5)
CUB AlexNet	adam	0.0001	64	224	20	StepLR(step=8, γ=0.5)
CUB VGG-16	adam	0.0001	64	224	20	StepLR(step=8, γ=0.5)
CUB ViT-B/16	adam	0.0001	16	224	20	StepLR(step=8, γ=0.5)

表 1 各实验超参数配置

1.5 实验步骤

1. 数据准备

下载并解压 CIFAR-10（torchvision 自动下载）和 CUB-200-2011 数据集，按照官方 train_test_split.txt 划分训练集与测试集，构建 PyTorch DataLoader。

2. 配置实验参数

为每个模型编写独立的 YAML 配置文件（位于 configs/ 目录），通过 --config 命令行参数传入统一的训练入口 train.py，实现超参数与代码的解耦。

3. 模型搭建与初始化

根据配置中的 model 字段实例化对应网络，替换分类头，并按需加载 ImageNet 预训练权重。SENet 因结构修改不加载预训练权重，全部随机初始化。

4. 训练与验证

在每个 Epoch 结束后计算验证集 Top-1 准确率和 Loss，保存最优权重（best_model.pth），同时生成训练曲线（training_curves.png）和混淆矩阵（best_confusion_matrix.png）。训练完成后输出 summary.json 汇总最优指标。

5. 结果分析

对比各模型在两个数据集上的表现，结合训练曲线和混淆矩阵，分析模型的收敛速度、泛化能力和易混淆类别分布。

1.6 实验结果与分析

1.6.1 CIFAR-10 总体结果

模型	最佳 Top-1 准确率	验证集 Loss	最优 Epoch
ResNet-18	86.41%	0.4098	10
SENet（手写实现）	86.01%	0.4100	8
AlexNet	72.35%	0.7977	10
MobileNetV3-Small	70.70%	0.8356	10
VGG-16	10.00%（未收敛）	2.3026	1

表 2 CIFAR-10 各模型性能对比

ResNet-18 以 86.41% 的准确率在 CIFAR-10 上取得最优，手写 SENet 以 86.01% 紧随其后，两者准确率相当但 SENet 在第 8 个 Epoch 即达到最优，收敛速度更快。AlexNet 和 MobileNet 分别止步于 72% 和 70%，前者网络容量有限，后者深度可分离卷积在小尺寸图像上的感受野优势未能充分发挥。VGG-16 在整个训练过程中准确率维持在随机水平（10%），Loss 未下降，推测原因是深层网络在缺少 BatchNorm（或适配的学习率）的情况下，在小尺寸 CIFAR 图像上出现梯度消失。

1.6.2 CIFAR-10 训练曲线

（1）ResNet-18

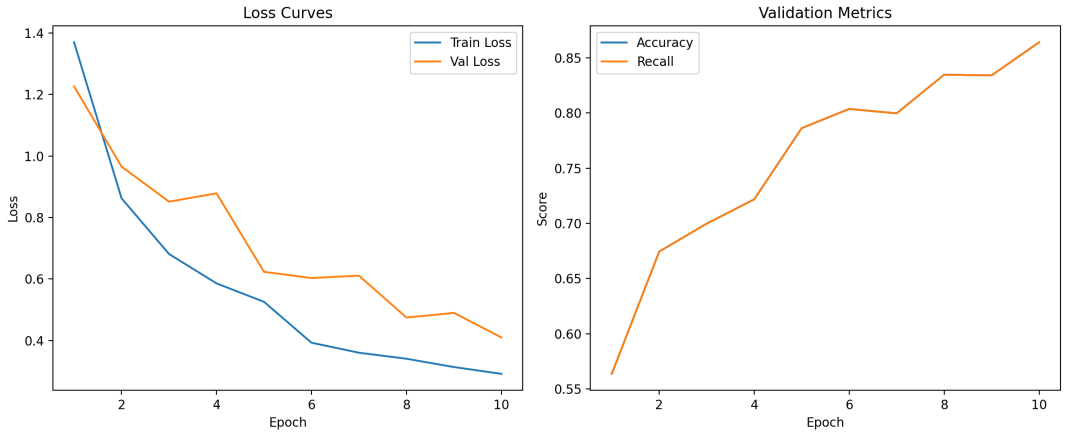

图 1 CIFAR-10 ResNet-18 训练/验证 Loss 与 Accuracy 曲线

（2）SENet（手写实现）

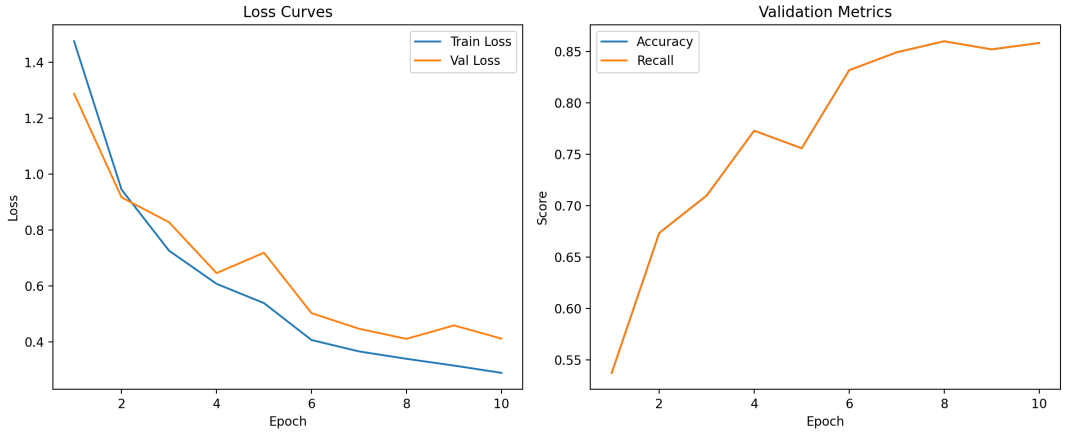

图 2 CIFAR-10 SENet 训练/验证 Loss 与 Accuracy 曲线

（3）AlexNet

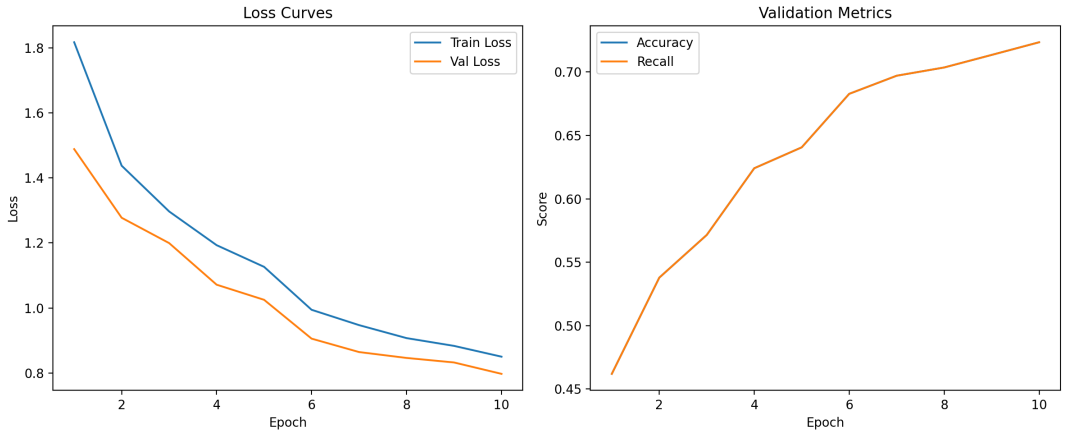

图 3 CIFAR-10 AlexNet 训练/验证 Loss 与 Accuracy 曲线

（4）MobileNetV3-Small

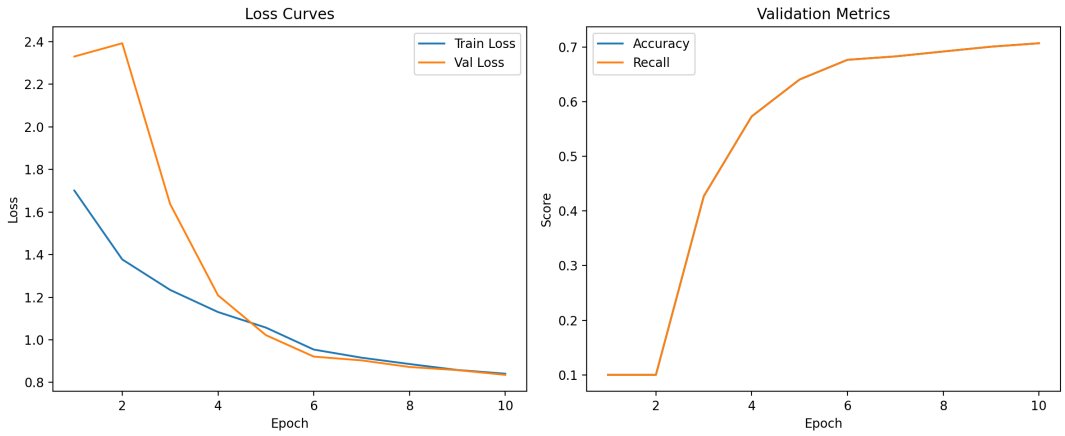

图 4 CIFAR-10 MobileNetV3-Small 训练/验证 Loss 与 Accuracy 曲线

（5）VGG-16

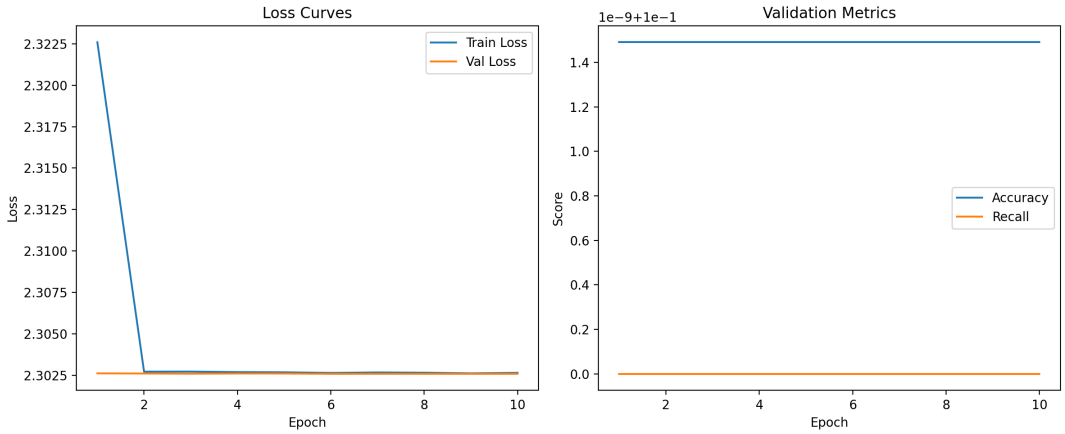

图 5 CIFAR-10 VGG-16 训练/验证 Loss 与 Accuracy 曲线（未收敛）

1.6.3 CIFAR-10 混淆矩阵

（1）ResNet-18

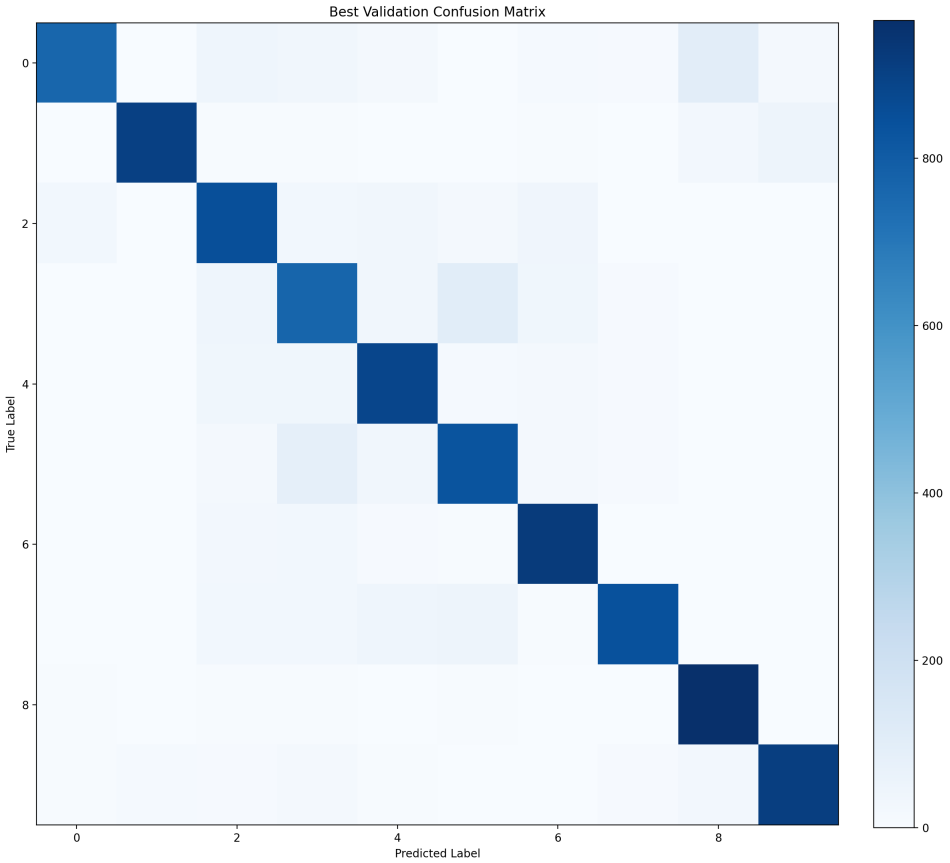

图 6 CIFAR-10 ResNet-18 混淆矩阵

（2）SENet（手写实现）

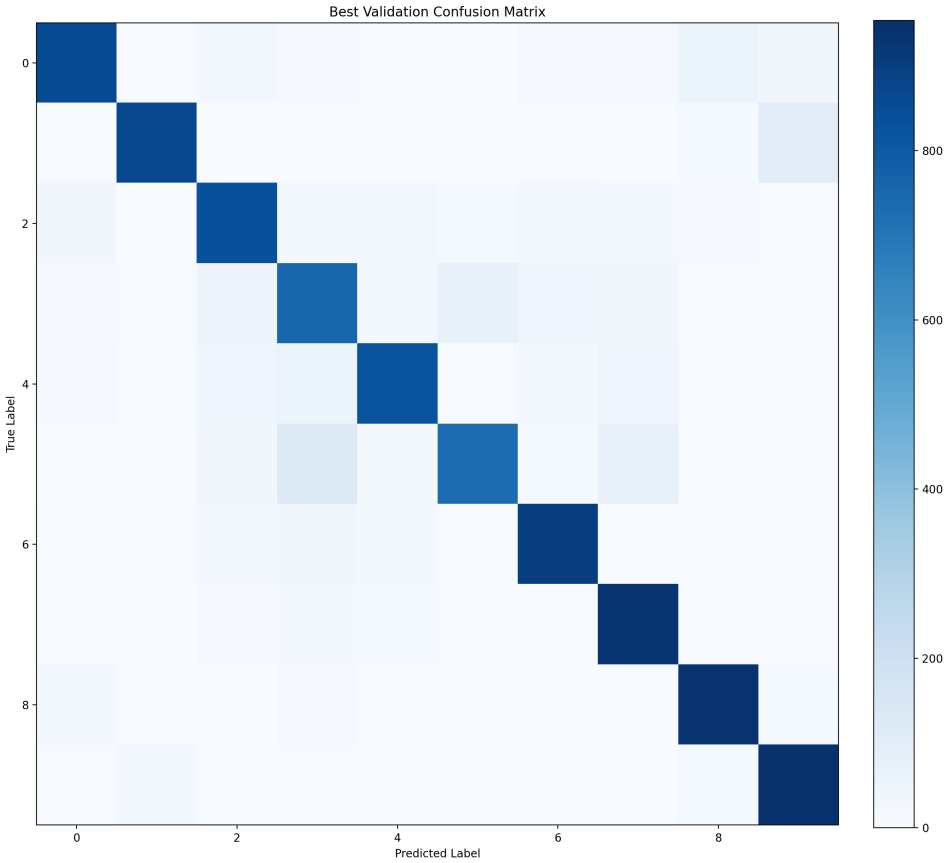

图 7 CIFAR-10 SENet 混淆矩阵

（3）AlexNet

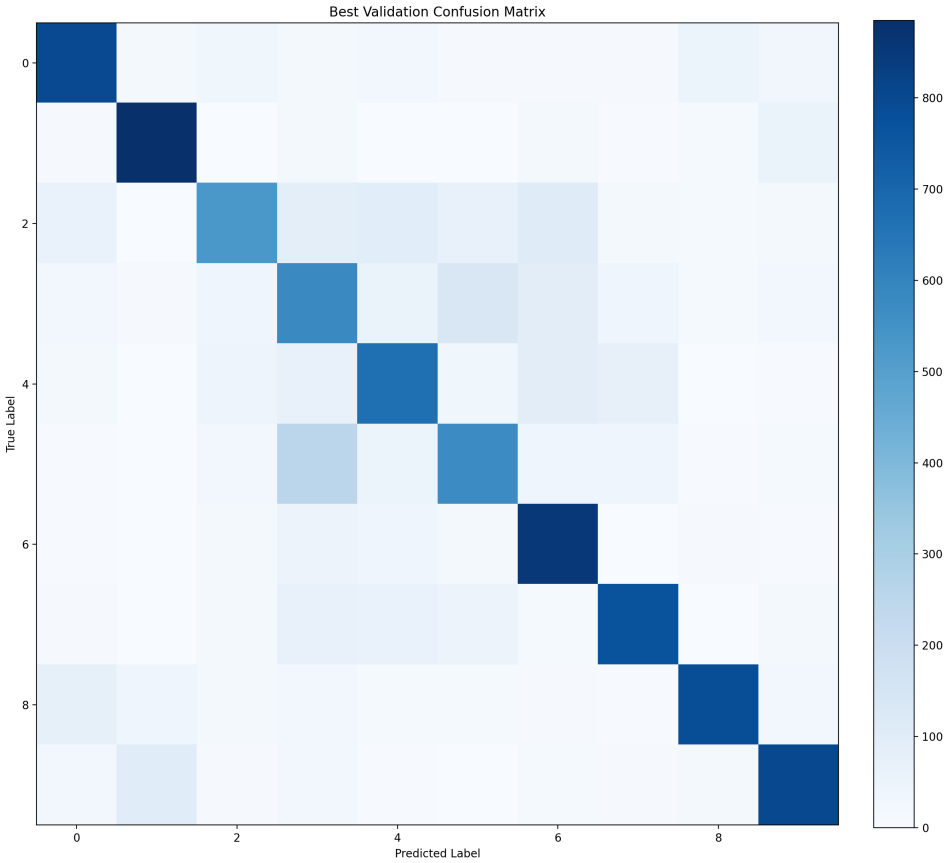

图 8 CIFAR-10 AlexNet 混淆矩阵

（4）MobileNetV3-Small

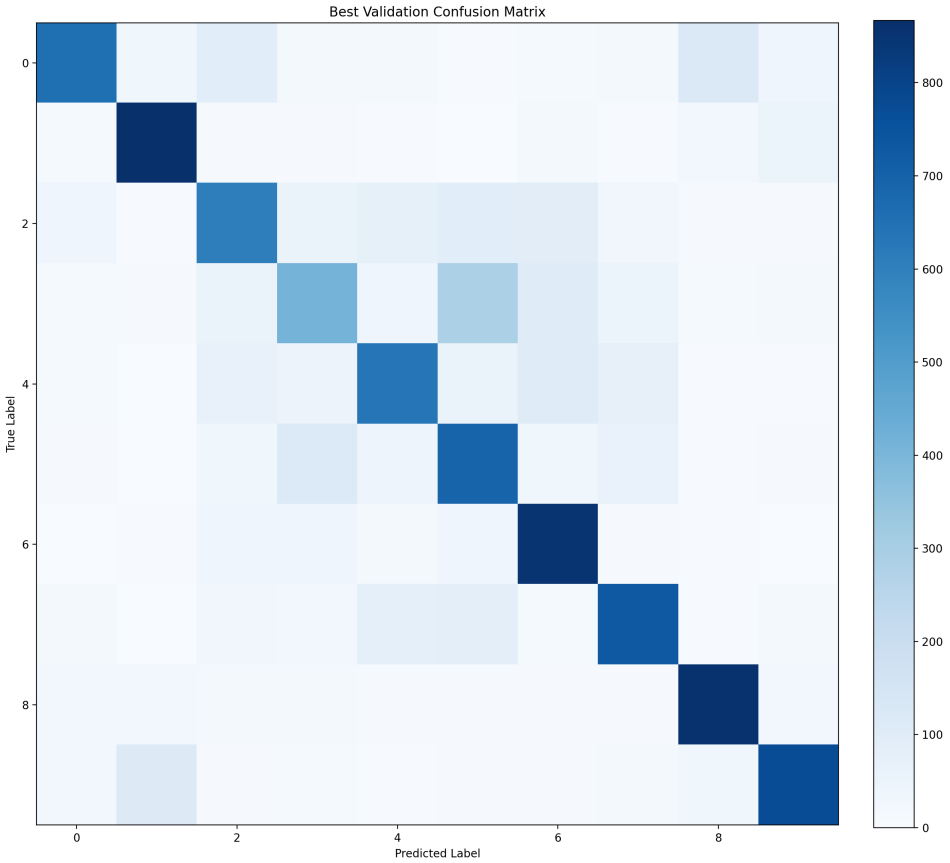

图 9 CIFAR-10 MobileNetV3-Small 混淆矩阵

（5）VGG-16

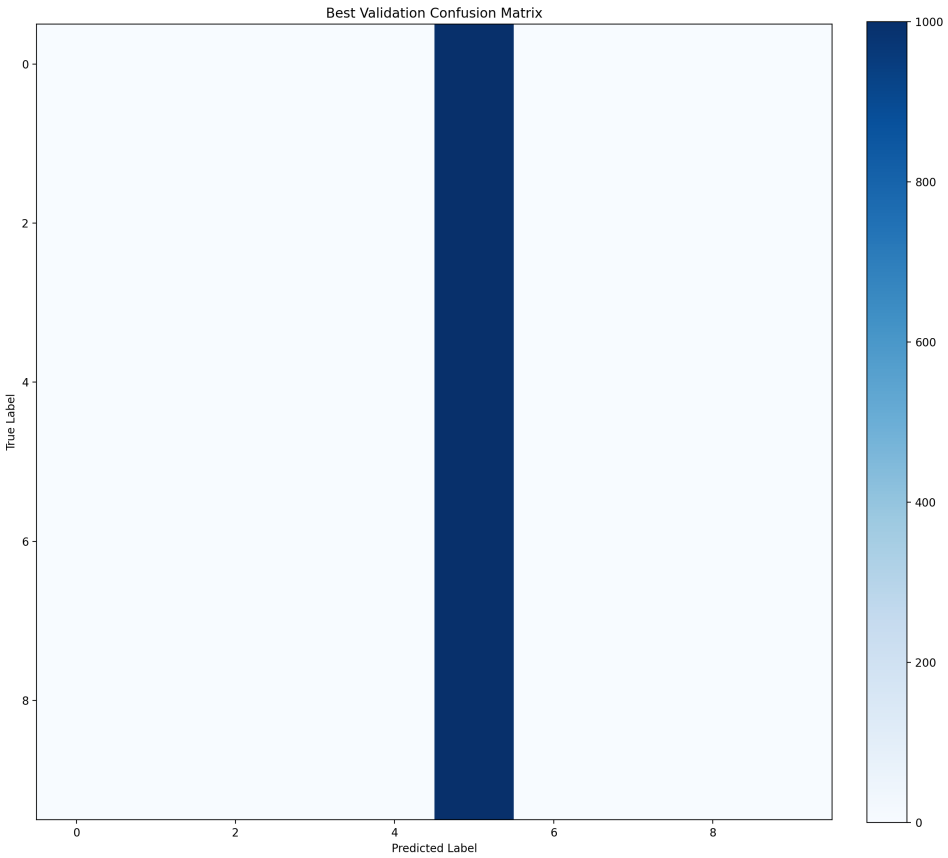

图 10 CIFAR-10 VGG-16 混淆矩阵

（6）ViT-B/16

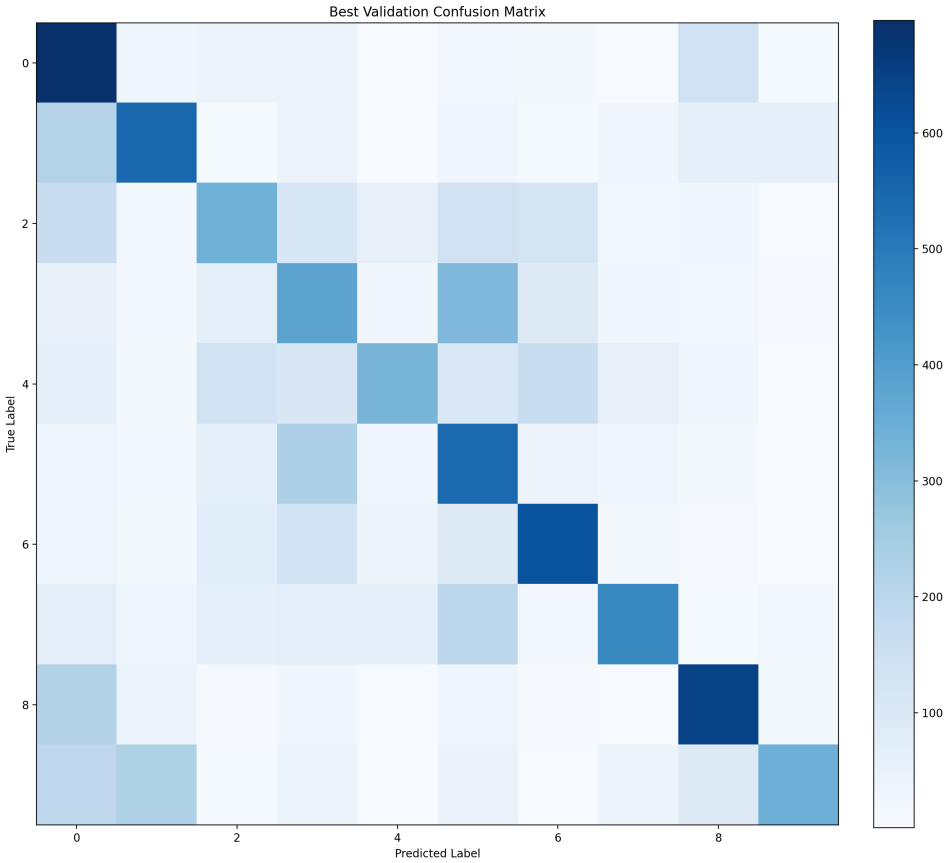

图 11 CIFAR-10 ViT-B/16 混淆矩阵

从 CIFAR-10 混淆矩阵可以看出，ResNet-18 和 SENet 在猫（cat）与狗（dog）、汽车（automobile）与卡车（truck）等视觉相似类别上仍有一定误判，这是 CIFAR-10 数据集固有的难点。VGG-16 几乎将所有样本预测为同一类别，印证了其未能收敛的结论。

1.6.4 CUB-200-2011 总体结果

模型	最佳 Top-1 准确率	最优 Epoch
SENet（手写实现）	30.45%	19
ResNet-50	18.66%	19
ViT-B/16	14.26%	20
AlexNet	12.24%	20
VGG-16	0.52%（未收敛）	1

表 3 CUB-200-2011 各模型性能对比

在 CUB 细粒度分类任务上，手写 SENet 以 30.45% 的准确率排名第一，大幅领先 ResNet-50（18.66%）和 ViT-B/16（14.26%）。这一结果出乎预期——通常参数量更大的 ResNet-50 和 ViT 应具备更强的表征能力，但 SENet 的 SE 通道注意力机制能够动态放大对鸟类细微辨别特征（如冠羽颜色、喙形、翼斑等）敏感的通道，在训练数据有限的情况下比纯靠深度叠加更具优势。ViT 需要大量数据预热 Patch Embedding，在仅 20 个 Epoch 的训练下远未达到最优。VGG-16 同样未能收敛（0.52%）。

1.6.5 CUB-200-2011 训练曲线

（1）SENet（手写实现）

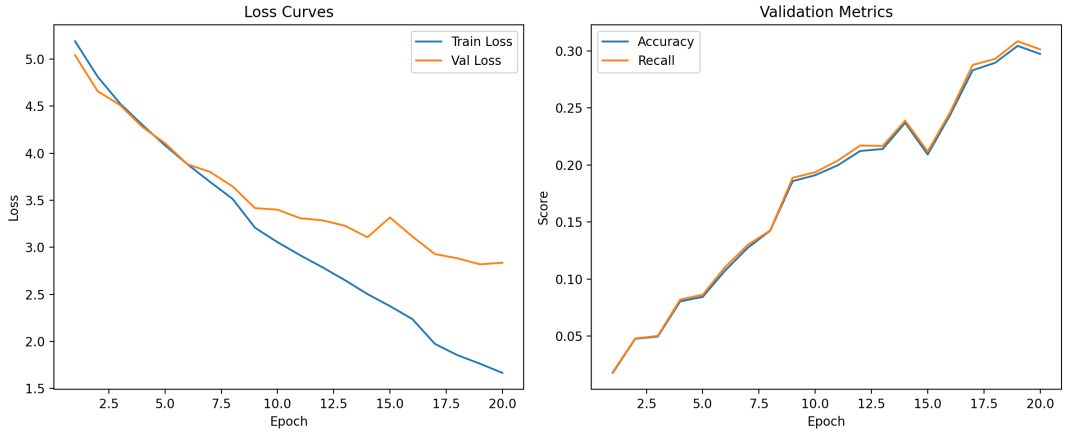

图 12 CUB SENet 训练/验证 Loss 与 Accuracy 曲线

（2）ResNet-50

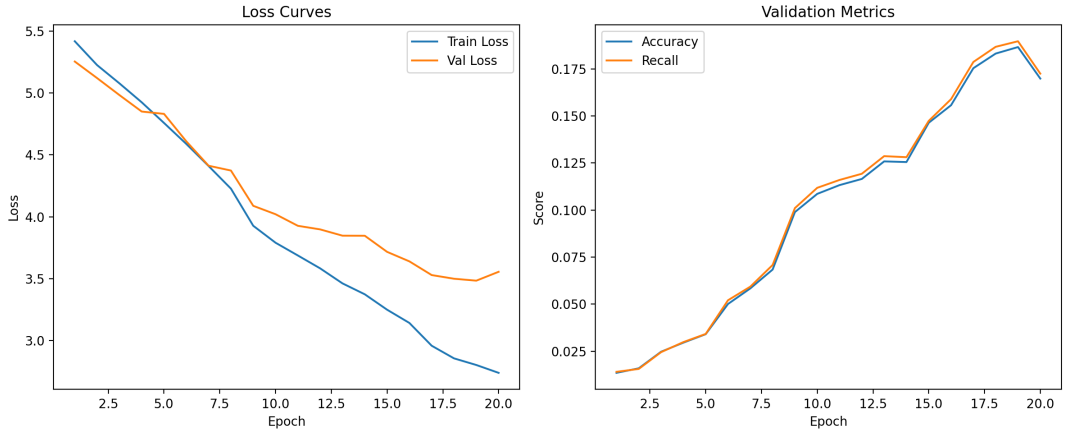

图 13 CUB ResNet-50 训练/验证 Loss 与 Accuracy 曲线

（3）ViT-B/16

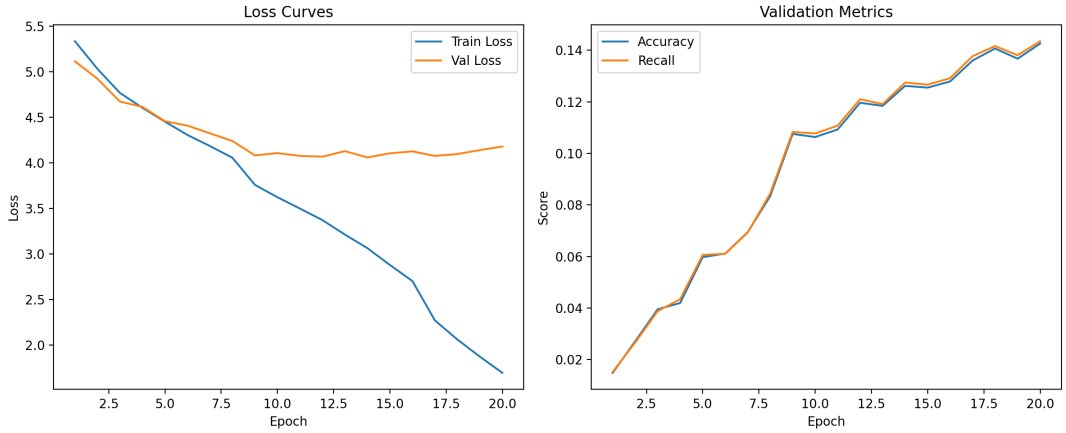

图 14 CUB ViT-B/16 训练/验证 Loss 与 Accuracy 曲线

（4）AlexNet

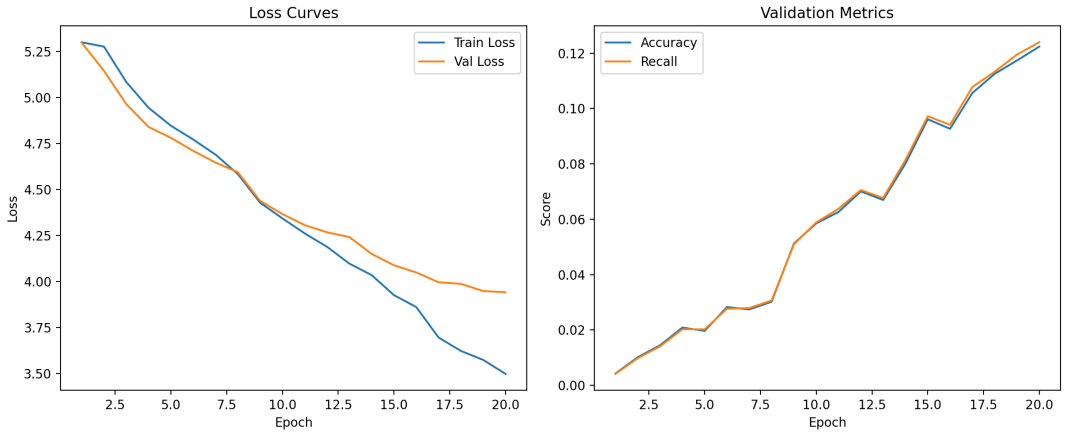

图 15 CUB AlexNet 训练/验证 Loss 与 Accuracy 曲线

（5）VGG-16

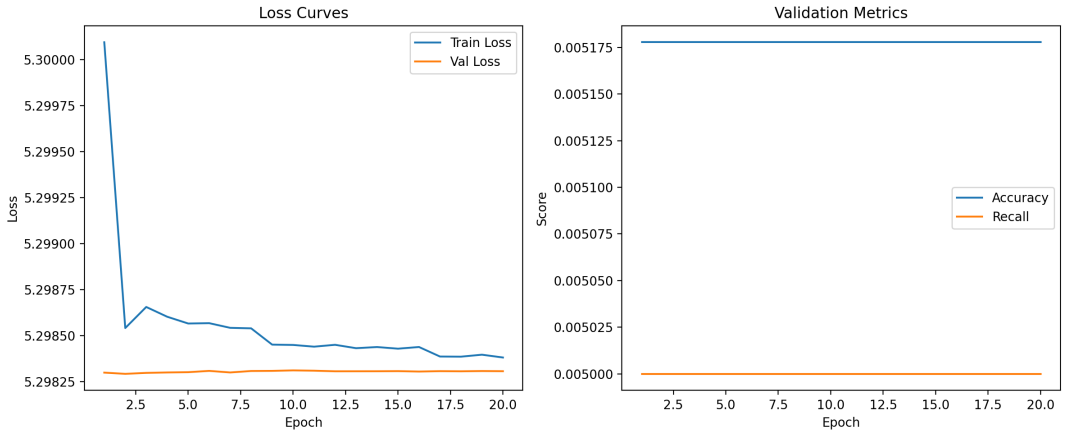

图 16 CUB VGG-16 训练/验证 Loss 与 Accuracy 曲线（未收敛）

1.6.6 CUB-200-2011 混淆矩阵

CUB 共 200 个细粒度类别，混淆矩阵规模较大（200×200），此处展示各模型的混淆矩阵以直观反映类别间混淆分布。对角线越亮、越集中，说明分类精度越高。

（1）SENet（手写实现）

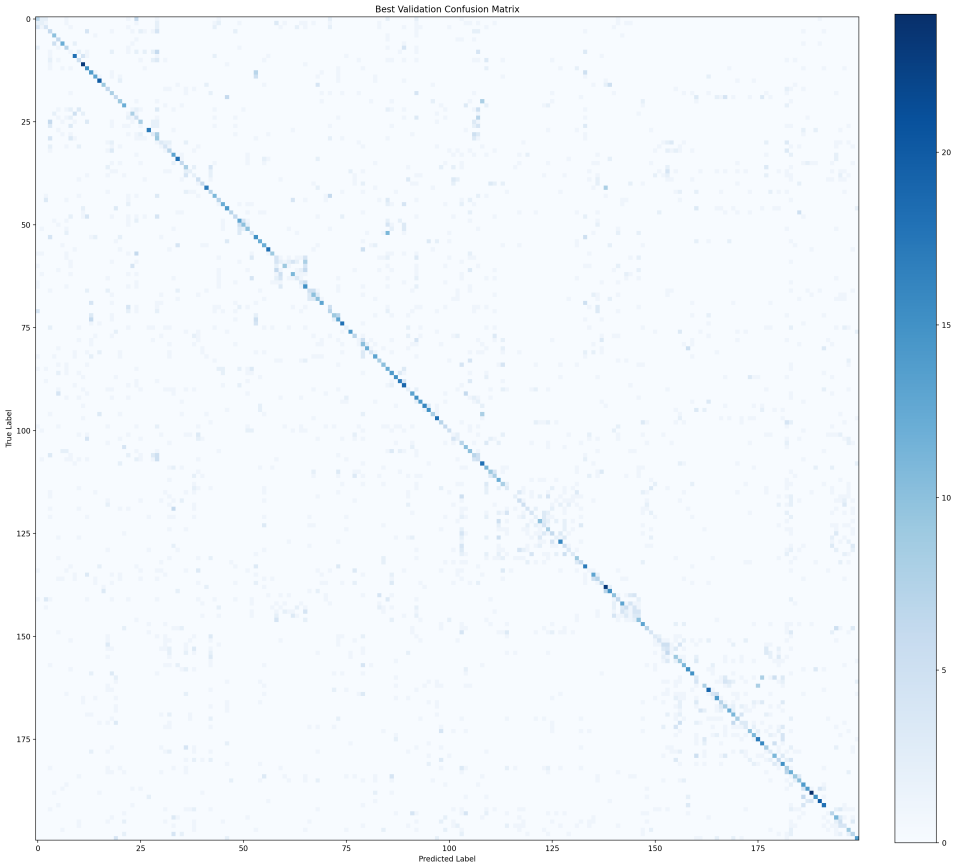

图 17 CUB SENet 混淆矩阵（200×200）

（2）ResNet-50

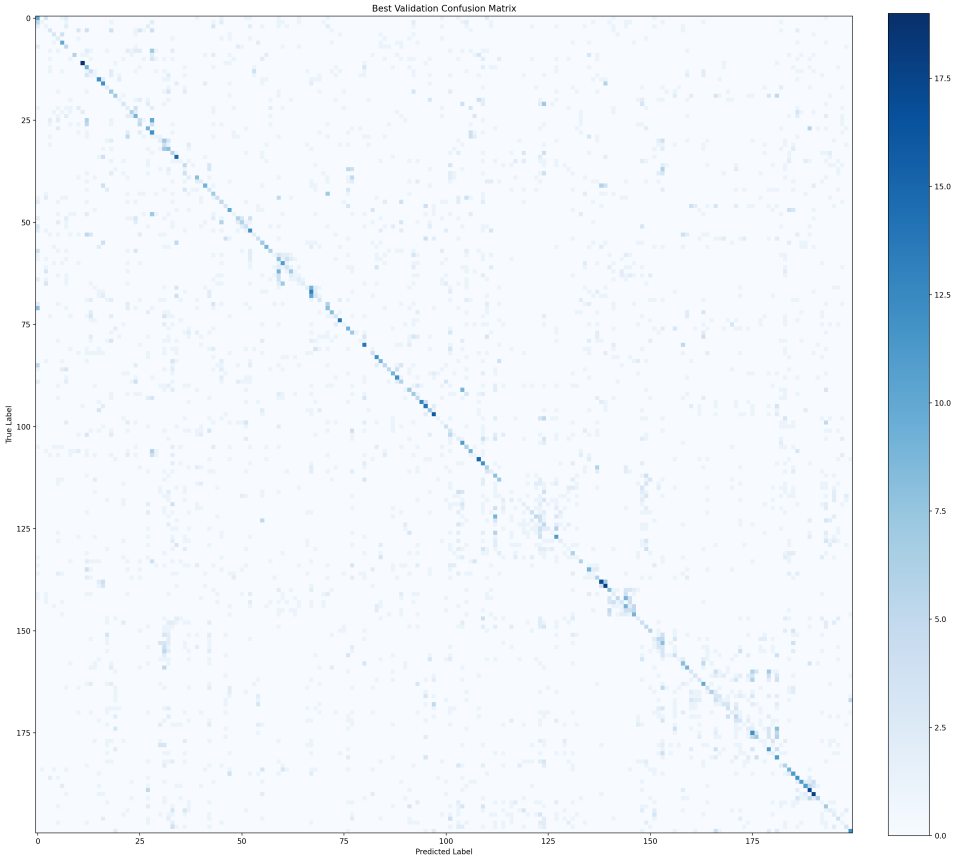

图 18 CUB ResNet-50 混淆矩阵（200×200）

（3）ViT-B/16

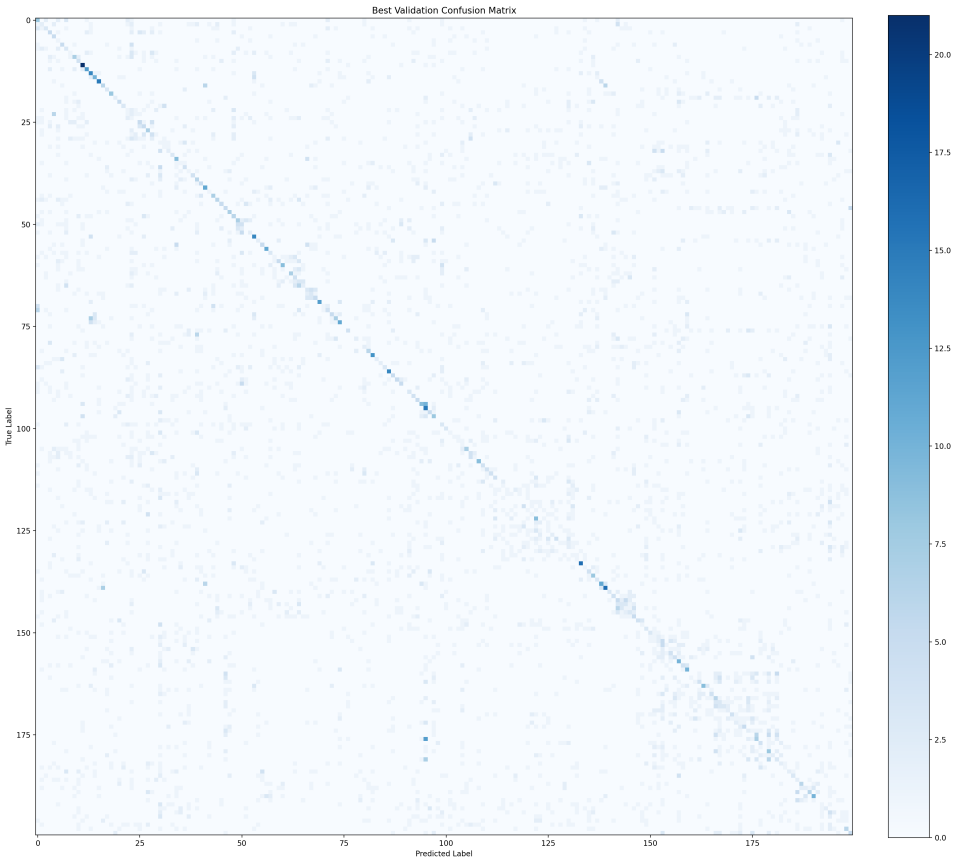

图 19 CUB ViT-B/16 混淆矩阵（200×200）

（4）AlexNet

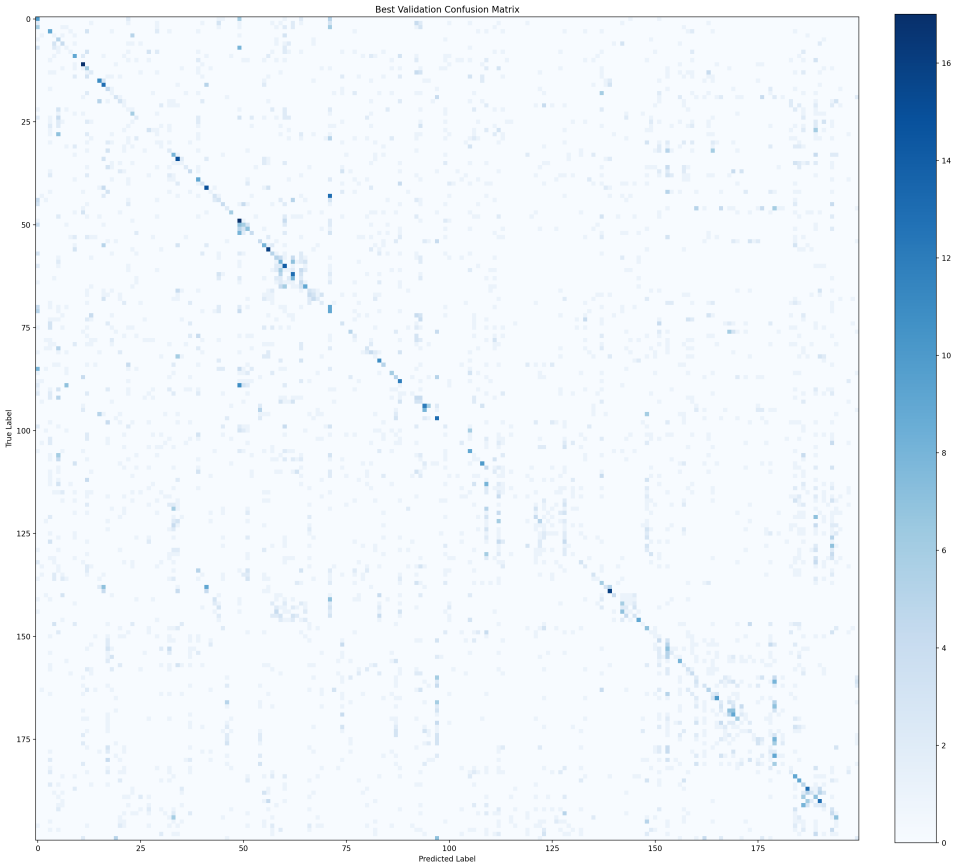

图 20 CUB AlexNet 混淆矩阵（200×200）

（5）VGG-16

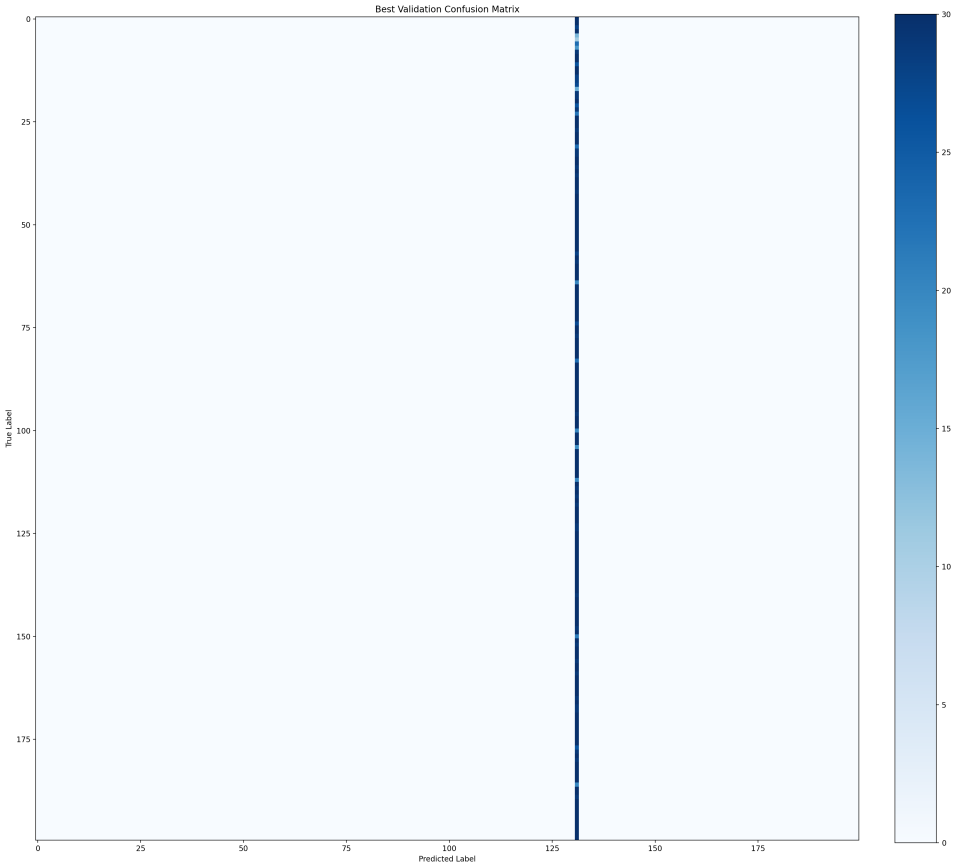

图 21 CUB VGG-16 混淆矩阵（200×200）

SENet 的混淆矩阵对角线最为清晰，说明其在各类别上的区分能力最强。ViT 的混淆矩阵较为弥散，反映出训练轮次不足导致的欠拟合。VGG-16 的混淆矩阵几乎全集中在某几列，与随机猜测结果相符。

1.7 实验总结

本实验系统性地对比了六种主流深度学习模型在 CIFAR-10 和 CUB-200-2011 数据集上的分类性能，重点验证了手动实现的 SE 注意力机制的有效性。

在 CIFAR-10 通用分类任务中，带残差连接的 ResNet-18 表现最优（86.41%），手写 SENet（86.01%）以更少的 Epoch 逼近同等精度，证明了 SE 通道注意力在加速收敛方面的作用。

在 CUB-200-2011 细粒度分类任务中，手写 SENet（30.45%）显著超越参数量更大的 ResNet-50（18.66%）和 ViT-B/16（14.26%），充分说明：在训练数据规模有限的场景下，注意力机制通过对关键通道的动态加权，能够更高效地利用训练信号，是解决细粒度识别问题的有效手段。

VGG-16 在两个数据集上均未收敛，提示其对学习率和 BatchNorm 配置的敏感性，后续可通过引入权重初始化策略或更小的初始学习率加以改善。